Rezolvări complete — Seminar 4

Semantica axiomatică. Enunţuri Hoare

Fundamentele limbajelor de programare

Semestrul II, 2025/2026

Scop. Pentru fiecare exerciţiu arătăm că enunţul Hoare cerut este demonstrabil, adică are forma

$$\vdash \{A\}\, c\, \{B\}.$$

La exerciţiile cu **while**, ideea centrală este alegerea unui **invariant**. Apoi demonstrăm trei lucruri:

1. precondiţia implică invariantul;
2. corpul buclei păstrează invariantul, presupunând că testul buclei este adevărat;
3. invariantul împreună cu negarea testului implică postcondiţia.

Cuprins

1 Regulile Hoare folosite **3**

2 Exerciţiul 1 **4**

2.1 Prima atribuire: $X := X + Y$ 4

2.2 A doua atribuire: $Y := X - Y$ 4

2.3 A treia atribuire: $X := X - Y$ 5

2.4 Concluzia prin secvenţiere 5

3 Exerciţiul 2 **6**

3.1 Invariantul 6

3.2 Precondiţia implică invariantul 6

3.3 Corpul păstrează invariantul 6

3.4 Aplicarea regulii pentru while 7

3.5 Postcondiţia 8

4 Exerciţiul 3 **9**

4.1 Invariantul 9

4.2 Precondiţia implică invariantul 9

4.3 Corpul păstrează invariantul 9

4.4 Aplicarea regulii pentru while 10

4.5 Postcondiţia 10

5 Exerciţiul 4 **11**

5.1 Invariantul 11
5.2 Precondiţia implică invariantul 11
5.3 Corpul păstrează invariantul 11
5.4 Aplicarea regulii pentru while 13
5.5 Postcondiţia 13

6 Rezumatul invarianţilor **14**

1 Regulile Hoare folosite

$$\frac{}{\{A\}\,\textbf{skip}\,\{A\}}\ \text{SKIP} \qquad \frac{}{\{B[X := a]\}\,X := a\,\{B\}}\ \text{ATRIB}$$

$$\frac{\{A\}\,c_0\,\{C\} \qquad \{C\}\,c_1\,\{B\}}{\{A\}\,c_0;\ c_1\,\{B\}}\ \text{SEQ} \qquad \frac{\{A \wedge b\}\,c_0\,\{B\} \qquad \{A \wedge \neg b\}\,c_1\,\{B\}}{\{A\}\,\textbf{if}\ b\ \textbf{then}\ c_0\ \textbf{else}\ c_1\,\{B\}}\ \text{IF}$$

$$\frac{\{A \wedge b\}\,c\,\{A\}}{\{A\}\,\textbf{while}\ b\ \textbf{do}\ c\,\{A \wedge \neg b\}}\ \text{WHILE} \qquad \frac{\models A \to A' \qquad \{A'\}\,c\,\{B'\} \qquad \models B' \to B}{\{A\}\,c\,\{B\}}\ \text{CONSEC}$$

Cum citim regula de atribuire. Pentru a demonstra $\vdash \{A\}\,X := a\,\{B\}$, regula directă ne cere să verificăm că A implică precondiţia calculată *înapoi*:

$$B[X := a].$$

Adică în postcondiţia dorită B înlocuim variabila modificată X cu expresia atribuită a.

Convenţie. Când scriem

$$A \Longrightarrow B,$$

înseamnă că folosim un fapt aritmetic valid, adică $\models A \to B$, deci putem aplica regula consecinţei. Acesta este exact stilul folosit la curs: partea de programare se face cu regulile Hoare, iar algebrei simple i se dă drumul prin regula consecinţei.

2 Exerciţiul 1

De demonstrat:

$$\vdash \{X = n \wedge Y = m\}\,(X := X + Y;\ Y := X - Y);\ X := X - Y\,\{X = m \wedge Y = n\}.$$

Programul este varianta clasică de interschimbare a valorilor fără variabilă auxiliară. Alegem aserţiunile intermediare:

$$\begin{aligned} A_0 &:= X = n \wedge Y = m, \\ A_1 &:= X = n + m \wedge Y = m, \\ A_2 &:= X = n + m \wedge Y = n, \\ A_3 &:= X = m \wedge Y = n. \end{aligned}$$

Vrem să legăm programul astfel:

$$\{A_0\}\,X := X + Y\,\{A_1\}, \qquad \{A_1\}\,Y := X - Y\,\{A_2\}, \qquad \{A_2\}\,X := X - Y\,\{A_3\}.$$

Apoi aplicăm de două ori regula de secvenţiere.

2.1 Prima atribuire: $X := X + Y$

Prin regula de atribuire, pentru postcondiţia A_1 trebuie să calculăm:

$$A_1[X := X + Y] \equiv X + Y = n + m \wedge Y = m.$$

Din A_0, adică din $X = n \wedge Y = m$, rezultă imediat

$$X + Y = n + m \wedge Y = m.$$

Deci, prin ATRIB şi apoi CONSEC, obţinem

$$\vdash \{A_0\}\,X := X + Y\,\{A_1\}.$$

2.2 A doua atribuire: $Y := X - Y$

Pentru postcondiţia A_2, regula de atribuire cere:

$$A_2[Y := X - Y] \equiv X = n + m \wedge X - Y = n.$$

Din A_1, adică din $X = n + m \wedge Y = m$, avem

$$X - Y = (n + m) - m = n.$$

Deci

$$A_1 \Longrightarrow A_2[Y := X - Y].$$

Prin ATRIB şi CONSEC,

$$\vdash \{A_1\}\,Y := X - Y\,\{A_2\}.$$

2.3 A treia atribuire: $X := X - Y$

Pentru postcondiţia A_3, avem:

$$A_3[X := X - Y] \equiv X - Y = m \wedge Y = n.$$

Din A_2, adică din $X = n + m \wedge Y = n$, rezultă

$$X - Y = (n + m) - n = m,$$

deci

$$A_2 \Longrightarrow A_3[X := X - Y].$$

Prin ATRIB şi CONSEC,

$$\vdash \{A_2\}\, X := X - Y\, \{A_3\}.$$

2.4 Concluzia prin secvenţiere

Punând cele trei bucăţi una după alta, obţinem:

$$\vdash \{X = n \wedge Y = m\}\, (X := X + Y;\ Y := X - Y);\ X := X - Y\, \{X = m \wedge Y = n\}.$$

3 Exerciţiul 2

De demonstrat:

$$\vdash \{I = 1 \land 1 \leq N \land S = 0\} \textbf{ while } I \leq 2 * N \textbf{ do } S := S + I;\ I := I + 2\,\{S = N * N\}.$$

Intuiţia este că programul adună primele N numere impare:

$$1 + 3 + 5 + \cdots + (2N - 1) = N^2.$$

Variabila I parcurge valorile impare, iar S reţine suma acumulată.

3.1 Invariantul

Alegem invariantul:

$$\mathcal{I} := \exists K \big(0 \leq K \land K \leq N \land I = 2 * K + 1 \land S = K * K\big).$$

Interpretarea este:

- K este numărul de iteraţii deja terminate;
- după K iteraţii, următorul impar este $I = 2K + 1$;
- suma deja strânsă este $S = K^2$.

3.2 Precondiţia implică invariantul

Presupunem

$$I = 1 \land 1 \leq N \land S = 0.$$

Luăm $K = 0$. Atunci:

$$0 \leq K, \qquad K \leq N, \qquad I = 1 = 2 * 0 + 1, \qquad S = 0 = 0 * 0.$$

Deci

$$I = 1 \land 1 \leq N \land S = 0 \Longrightarrow \mathcal{I}.$$

3.3 Corpul păstrează invariantul

Trebuie să demonstrăm:

$$\vdash \{\mathcal{I} \land I \leq 2 * N\}\, S := S + I;\ I := I + 2\,\{\mathcal{I}\}.$$

Folosim aserţiunea intermediară

$$B := \exists K \big(0 \leq K \land K < N \land I = 2 * K + 1 \land S = (K + 1) * (K + 1)\big).$$

Aceasta descrie situaţia de după $S := S + I$, dar înainte de $I := I + 2$: suma a fost actualizată, însă I este încă vechiul impar.

Pasul 1: $S := S + I$

Presupunem $\mathcal{I} \wedge I \leq 2 * N$. Atunci există K cu

$$0 \leq K, \qquad K \leq N, \qquad I = 2K + 1, \qquad S = K^2.$$

Din $I \leq 2N$ şi $I = 2K + 1$, rezultă

$$2K + 1 \leq 2N,$$

deci, peste numere întregi,

$$K < N.$$

După atribuirea $S := S + I$, noua valoare a lui S este

$$S + I = K^2 + (2K + 1) = (K + 1)^2.$$

Deci precondiţia implică exact precondiţia calculată înapoi pentru B:

$$\mathcal{I} \wedge I \leq 2N \Longrightarrow B[S := S + I].$$

Prin ATRIB şi CONSEC,

$$\vdash \{\mathcal{I} \wedge I \leq 2 * N\}\, S := S + I\, \{B\}.$$

Pasul 2: $I := I + 2$

Presupunem B. Există K cu

$$0 \leq K, \qquad K < N, \qquad I = 2K + 1, \qquad S = (K + 1)^2.$$

După $I := I + 2$, noua valoare a lui I este

$$I + 2 = 2K + 3 = 2(K + 1) + 1.$$

Cum $K < N$, avem $K + 1 \leq N$. Aşadar, alegând $K' := K + 1$, obţinem invariantul:

$$0 \leq K', \qquad K' \leq N, \qquad I_{nou} = 2K' + 1, \qquad S = K'^2.$$

Deci

$$B \Longrightarrow \mathcal{I}[I := I + 2].$$

Prin ATRIB şi CONSEC,

$$\vdash \{B\}\, I := I + 2\, \{\mathcal{I}\}.$$

Prin secvenţiere,

$$\vdash \{\mathcal{I} \wedge I \leq 2 * N\}\, S := S + I;\ I := I + 2\, \{\mathcal{I}\}.$$

3.4 Aplicarea regulii pentru while

Din rezultatul anterior, prin regula pentru while,

$$\vdash \{\mathcal{I}\}\ \textbf{while}\ I \leq 2 * N\ \textbf{do}\ S := S + I;\ I := I + 2\, \{\mathcal{I} \wedge \neg(I \leq 2 * N)\}.$$

3.5 Postcondiţia

Presupunem $\mathcal{I} \wedge \neg(I \leq 2N)$. Din invariant există K cu

$$0 \leq K, \qquad K \leq N, \qquad I = 2K + 1, \qquad S = K^2.$$

Din $\neg(I \leq 2N)$, avem $I > 2N$. Cu $I = 2K + 1$, rezultă

$$2K + 1 > 2N.$$

Dacă am avea $K < N$, atunci $K \leq N - 1$, deci

$$2K + 1 \leq 2N - 1 < 2N,$$

contradicţie. Prin urmare $K = N$. Cum $S = K^2$, obţinem

$$S = N^2.$$

Deci

$$\mathcal{I} \wedge \neg(I \leq 2N) \Longrightarrow S = N * N.$$

Aplicând regula consecinţei la început şi la final, obţinem enunţul cerut:

$$\vdash \{I = 1 \wedge 1 \leq N \wedge S = 0\}\ \textbf{while}\ I \leq 2 * N\ \textbf{do}\ S := S + I;\ I := I + 2\ \{S = N * N\}.$$

4 Exerciţiul 3

De demonstrat:

$$\vdash \{1 \leq N \wedge P = 0 \wedge C = 1\} \textbf{ while } C \leq N \textbf{ do } P := P + M;\ C := C + 1 \{P = M * N\}.$$

Programul adună valoarea M de N ori. Variabila C este contorul, iar P este produsul acumulat prin adunări repetate.

4.1 Invariantul

Alegem invariantul:

$$\mathcal{I} := 1 \leq N \wedge 1 \leq C \wedge C \leq N + 1 \wedge P = M * (C - 1).$$

Interpretare:

- C spune la ce pas suntem;
- până la pasul C, am adunat deja M de $C - 1$ ori;
- deci $P = M * (C - 1)$.

4.2 Precondiţia implică invariantul

Din

$$1 \leq N \wedge P = 0 \wedge C = 1$$

obţinem:

$$1 \leq N, \qquad 1 \leq C, \qquad C = 1 \leq N + 1, \qquad P = 0 = M * (1 - 1) = M * (C - 1).$$

Deci

$$1 \leq N \wedge P = 0 \wedge C = 1 \Longrightarrow \mathcal{I}.$$

4.3 Corpul păstrează invariantul

Trebuie să demonstrăm:

$$\vdash \{\mathcal{I} \wedge C \leq N\}\, P := P + M;\ C := C + 1\, \{\mathcal{I}\}.$$

Folosim aserţiunea intermediară

$$B := 1 \leq N \wedge 1 \leq C \wedge C \leq N \wedge P = M * C.$$

Ea descrie situaţia după $P := P + M$: am adunat M încă o dată, dar contorul C nu a fost încă incrementat.

Pasul 1: $P := P + M$

Din $\mathcal{I} \wedge C \leq N$ avem

$$P = M * (C - 1) \qquad \text{şi} \qquad C \leq N.$$

După atribuirea $P := P + M$, noul P este

$$P + M = M * (C - 1) + M = M * C.$$

Prin urmare

$$\mathcal{I} \wedge C \leq N \Longrightarrow B[P := P + M].$$

Deci, prin ATRIB şi CONSEC,

$$\vdash \{\mathcal{I} \wedge C \leq N\}\, P := P + M\, \{B\}.$$

Pasul 2: $C := C + 1$

Presupunem B. După $C := C + 1$, noul contor este $C + 1$. Trebuie să verificăm invariantul pentru noul C:

$$1 \leq C + 1, \qquad C + 1 \leq N + 1, \qquad P = M * ((C + 1) - 1) = M * C.$$

Toate urmează direct din B. Deci

$$B \Longrightarrow \mathcal{I}[C := C + 1].$$

Prin ATRIB şi CONSEC,

$$\vdash \{B\}\, C := C + 1\, \{\mathcal{I}\}.$$

Prin secvenţiere,

$$\vdash \{\mathcal{I} \wedge C \leq N\}\, P := P + M;\ C := C + 1\, \{\mathcal{I}\}.$$

4.4 Aplicarea regulii pentru while

Prin regula pentru while:

$$\vdash \{\mathcal{I}\}\, \textbf{while}\ C \leq N\ \textbf{do}\ P := P + M;\ C := C + 1\, \{\mathcal{I} \wedge \neg(C \leq N)\}.$$

4.5 Postcondiţia

Din $\mathcal{I} \wedge \neg(C \leq N)$ avem simultan:

$$C \leq N + 1 \qquad \text{şi} \qquad C > N.$$

Deoarece lucrăm peste numere întregi, rezultă

$$C = N + 1.$$

Din invariant,

$$P = M * (C - 1) = M * ((N + 1) - 1) = M * N.$$

Deci

$$\mathcal{I} \wedge \neg(C \leq N) \Longrightarrow P = M * N.$$

Aplicând regula consecinţei la început şi la final, concluzionăm:

$$\vdash \{1 \leq N \wedge P = 0 \wedge C = 1\}\, \textbf{while}\ C \leq N\ \textbf{do}\ P := P + M;\ C := C + 1\, \{P = M * N\}.$$

5 Exerciţiul 4

De demonstrat:

$\vdash \{I = 0 \wedge \exists K(N = K*K)\}$ **while** $I*I \leq N$ **do** (**if** $I*I = N$ **then** $P := 1$ **else skip**); $I := I+1$ $\{P = 1\}$.

Programul caută un I cu $I^2 = N$. Dacă îl găseşte, setează $P := 1$. Precondiţia spune că N este pătrat perfect, deci un asemenea I va fi atins de buclă.

5.1 Invariantul

Folosim invariantul:

$$\mathcal{I} := \exists K\big(0 \leq K \wedge N = K * K \wedge 0 \leq I \wedge (I \leq K \vee P = 1)\big).$$

Interpretare:

- K este o rădăcină nenegativă a lui N, deci $N = K^2$;
- I creşte de la 0 în sus;
- cât timp nu am trecut de K, avem $I \leq K$;
- dacă am trecut de K, atunci trebuie deja să avem $P = 1$.

Din $\exists K(N = K * K)$ putem obţine $\exists K(0 \leq K \wedge N = K * K)$: dacă un K satisface $N = K^2$, atunci fie $K \geq 0$, fie $-K \geq 0$, iar $(-K)^2 = K^2$. Aceasta este o implicaţie aritmetică folosită prin regula consecinţei.

5.2 Precondiţia implică invariantul

Presupunem

$$I = 0 \wedge \exists K(N = K * K).$$

Alegem o rădăcină nenegativă K cu $N = K * K$. Atunci:

$$0 \leq K, \qquad N = K * K, \qquad 0 \leq I, \qquad I = 0 \leq K.$$

Deci

$$I = 0 \wedge \exists K(N = K * K) \Longrightarrow \mathcal{I}.$$

5.3 Corpul păstrează invariantul

Corpul este

$$c := (\textbf{if } I * I = N \textbf{ then } P := 1 \textbf{ else skip});\ I := I + 1.$$

Trebuie să demonstrăm:

$$\vdash \{\mathcal{I} \wedge I * I \leq N\}\, c\, \{\mathcal{I}\}.$$

Folosim aserţiunea intermediară

$$B := \exists K\big(0 \leq K \wedge N = K * K \wedge 0 \leq I + 1 \wedge (I + 1 \leq K \vee P = 1)\big).$$

Aceasta este exact invariantul care trebuie să fie adevărat după incrementarea lui I, dar scris înainte de comanda $I := I + 1$.

Pasul 1: instrucţiunea if

Vrem:

$$\vdash \{\mathcal{I} \wedge I * I \leq N\}\, \textbf{if } I * I = N \textbf{ then } P := 1 \textbf{ else skip}\, \{B\}.$$

Prin regula pentru **if**, tratăm cele două ramuri.

Ramura adevărată: $I * I = N$**.** Trebuie:

$$\vdash \{\mathcal{I} \wedge I * I \leq N \wedge I * I = N\}\, P := 1\, \{B\}.$$

Prin regula de atribuire, precondiţia exactă pentru $P := 1$ este

$$B[P := 1].$$

Dar $B[P := 1]$ conţine disjuncţia

$$I + 1 \leq K \vee 1 = 1,$$

care este adevărată. Din invariant avem deja existenţa lui K cu $0 \leq K$, $N = K * K$, iar din $0 \leq I$ rezultă $0 \leq I + 1$. Deci

$$\mathcal{I} \wedge I * I \leq N \wedge I * I = N \Longrightarrow B[P := 1].$$

Prin ATRIB şi CONSEC,

$$\vdash \{\mathcal{I} \wedge I * I \leq N \wedge I * I = N\}\, P := 1\, \{B\}.$$

Ramura falsă: $\neg(I * I = N)$**.** Trebuie:

$$\vdash \{\mathcal{I} \wedge I * I \leq N \wedge \neg(I * I = N)\}\, \textbf{skip}\, \{B\}.$$

Din invariant alegem K cu

$$0 \leq K, \qquad N = K * K, \qquad 0 \leq I, \qquad I \leq K \vee P = 1.$$

Mai avem

$$I * I \leq N = K * K \qquad \text{şi} \qquad I * I \neq N = K * K.$$

Deci

$$I^2 < K^2.$$

Cum $0 \leq I$ şi $0 \leq K$, rezultă

$$I < K,$$

adică, peste numere întregi,

$$I + 1 \leq K.$$

Prin urmare B este adevărat. Cum **skip** păstrează aserţiunea, avem:

$$\vdash \{B\}\, \textbf{skip}\, \{B\}.$$

Aplicând consecinţa la precondiţie, obţinem:

$$\vdash \{\mathcal{I} \wedge I * I \leq N \wedge \neg(I * I = N)\}\, \textbf{skip}\, \{B\}.$$

Prin regula pentru **if**, rezultă:

$$\vdash \{\mathcal{I} \wedge I * I \leq N\}\, \textbf{if } I * I = N \textbf{ then } P := 1 \textbf{ else skip}\, \{B\}.$$

Pasul 2: instrucţiunea $I := I + 1$

Din definiţia lui B, observăm că

$$B \equiv \mathcal{I}[I := I + 1].$$

Aşadar, prin regula de atribuire,

$$\vdash \{B\}\, I := I + 1\, \{\mathcal{I}\}.$$

Prin secvenţiere,

$$\vdash \{\mathcal{I} \wedge I * I \leq N\}\, (\textbf{if } I * I = N \textbf{ then } P := 1 \textbf{ else skip});\ I := I + 1\, \{\mathcal{I}\}.$$

5.4 Aplicarea regulii pentru while

Prin regula pentru while,

$$\vdash \{\mathcal{I}\}\, \textbf{while } I * I \leq N \textbf{ do } (\textbf{if } I * I = N \textbf{ then } P := 1 \textbf{ else skip});\ I := I + 1\, \{\mathcal{I} \wedge \neg(I * I \leq N)\}.$$

5.5 Postcondiţia

Presupunem $\mathcal{I} \wedge \neg(I * I \leq N)$. Din invariant există K cu

$$0 \leq K, \qquad N = K * K, \qquad 0 \leq I, \qquad I \leq K \vee P = 1.$$

Din $\neg(I * I \leq N)$ şi $N = K * K$, avem

$$I^2 > K^2.$$

Cum $I \geq 0$ şi $K \geq 0$, rezultă

$$I > K.$$

Deci $I \leq K$ este fals. Din disjuncţia invariantului

$$I \leq K \vee P = 1$$

rezultă obligatoriu

$$P = 1.$$

Prin consecinţă, obţinem concluzia cerută:

$$\vdash \{I = 0 \wedge \exists K(N = K * K)\}\, \textbf{while } I * I \leq N \textbf{ do } (\textbf{if } I * I = N \textbf{ then } P := 1 \textbf{ else skip});\ I := I + 1\, \{P = 1\}.$$

6 Rezumatul invarianţilor

Exerciţiul 2:

$$\mathcal{I}_2 := \exists K \big(0 \leq K \wedge K \leq N \wedge I = 2 * K + 1 \wedge S = K * K\big).$$

Idee: după K paşi, suma primelor K numere impare este K^2.

Exerciţiul 3:

$$\mathcal{I}_3 := 1 \leq N \wedge 1 \leq C \wedge C \leq N + 1 \wedge P = M * (C - 1).$$

Idee: după $C - 1$ paşi, am adunat M de $C - 1$ ori.

Exerciţiul 4:

$$\mathcal{I}_4 := \exists K \big(0 \leq K \wedge N = K * K \wedge 0 \leq I \wedge (I \leq K \vee P = 1)\big).$$

Idee: dacă I nu a trecut încă de rădăcina K, atunci $I \leq K$; dacă a trecut, atunci programul a găsit deja $I^2 = N$ şi a setat $P = 1$.